

Laboratory Manual

DATABASE SYSTEMS LAB (CS – 229L)

4rd Semester Fall 2024



Name: Muhammad Faisal

Roll No: 2022F-BCS-152

Section: C - Alpha

BS in Information Technology

Department of Computer Science & Information Technology

Sir Syed University of Engineering and Technology

University Road Karachi - 75300

<http://www.ssuet.edu.pk>

Course Learning Outcome(s):

No.	Course Learning Outcomes (CLOs)	Student Outcomes (SOs)	Bloom's Taxonomy
1	Follow Instructions to specified problem definition for handling operations like searching, insertion, deletion, traversing mechanism on data structures.	PLO_5 (Modern Tool Usage)	P3 (Guided Response)
2	Construct a technical project which involves the entire knowledge of the course.	PLO_4 (Design/ Development of Solutions)	P4 (Mechanism)
3	Demonstrate the knowledge which will be assessed through viva and presentation.	PLO_7 (Communication)	A3 (Valuing)

Laboratory Experiments

Lab #	Topics	Page #
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		
11		
12		
13		
14		

Laboratory Rubrics

Criteria	Exceeds Expectations (≥90%)	Meets Expectations (70%-89%)	Developing (50%-69%)	Unsatisfactory (<50%)
Software Handling (2)	Able to use software with its standard and advanced features without assistance	Able to use software with its standard and advanced features with minimal assistance	Able to use software with its standard features with assistance	Unable to use the software
Programming/ Simulation (5)	Able to program/simulate the lab tasks with simplification	Able to program/simulate the lab tasks without errors	Able to program/simulate lab tasks with errors	Unable to program/simulate
Results (2)	Able to achieve all the desired results with alternate ways to improve measurements	Able to achieve all the desired results	Able to achieve most of the desired results with errors	Unable to achieve the desired results
Lab Report (1)	Laboratory manual has no grammatical and/or spelling errors. All sections of the report are very well written and technically accurate.	Laboratory manual has very few grammatical/spelling errors. All sections of the report are technically accurate.	-Laboratory manual has multiple grammatical/spelling errors. Few sections of the report contain technical errors.	Laboratory manual has several grammatical/spelling errors and sentence construction is poor. -All sections of the report contain multiple technical errors.

Laboratory Report Score

Lab#	Criteria				Score
1.	Software Handling ()/2	Programming/ Simulations()/5	Results ()/2	Lab Report ()/1	
2.	Software Handling ()/2	Programming/ Simulations()/5	Results ()/2	Lab Report ()/1	
3.	Software Handling ()/2	Programming/ Simulations()/5	Results ()/2	Lab Report ()/1	
4.	Software Handling ()/2	Programming/ Simulations()/5	Results ()/2	Lab Report ()/1	
5.	Software Handling ()/2	Programming/ Simulations()/5	Results ()/2	Lab Report ()/1	
6.	Software Handling ()/2	Programming/ Simulations()/5	Results ()/2	Lab Report ()/1	
7.	Software Handling ()/2	Programming/ Simulations()/5	Results ()/2	Lab Report ()/1	
8.	Software Handling ()/2	Programming/ Simulations()/5	Results ()/2	Lab Report ()/1	
9.	Software Handling ()/2	Programming/ Simulations()/5	Results ()/2	Lab Report ()/1	
10.	Software Handling ()/2	Programming/ Simulations()/5	Results ()/2	Lab Report ()/1	
11.	Software Handling ()/2	Programming/ Simulations()/5	Results ()/2	Lab Report ()/1	
12.	Software Handling ()/2	Programming/ Simulations()/5	Results ()/2	Lab Report ()/1	
13.	Software Handling ()/2	Programming/ Simulations()/5	Results ()/2	Lab Report ()/1	
14.	Software Handling ()/2	Programming/ Simulations()/5	Results ()/2	Lab Report ()/1	
Obtained Score out of (140)					
Obtained Score out of (X)= Obtained Score out of (140) /140* X					

Semester Project Rubric

Criteria	Exceeds Expectations (≥90%)	Meets Expectations (70%-89%)	Developing (50%-69%)	Unsatisfactory (<50%)	Score
Project Demonstration (5)	Able to demonstrate the project with achievement of required objectives having clear understanding of project limitations and future enhancements. Hardware and/or Software modules are fully functional, if applicable.	Able to demonstrate the project with achievement of required objectives but understanding of project limitations and future enhancements is insufficient. Hardware and/or Software modules are functional, if applicable.	Able to demonstrate the project with achievement of at least 50% required objectives and insufficient understanding of project limitations and future enhancements. Hardware and/or Software modules are partially functional, if applicable.	Able to demonstrate the project with achievement of less than 50% required objectives and lacks in understanding of project limitations and future enhancements. Hardware and/or Software modules are not functional, if applicable.	
Project results (3)	Able to achieve all the desired results with alternate ways to improve measurements	Able to achieve all the desired results	Able to achieve most of the desired results with errors	Unable to achieve the desired results	
Report Writing (3)	Project report has no grammatical and/ or spelling errors. All sections of the report are very well-written and technically accurate.	Project report has very few grammatical/ spelling errors. All sections of the report are technically accurate.	Project report has multiple grammatical/ spelling errors. Few sections of the report contain technical errors.	Project report has several grammatical/ spelling errors and sentence construction is poor.	
Viva (4)	Able to answer the questions easily and correctly across the project.	Able to answer the questions related to the project	Able to answer the questions but with mistakes	Unable to answer the questions	
Obtained Score out of (15)					
Obtained Score out of (X)= Obtained Score out of (15) /15* X					

Laboratory Examination Rubric

Criteria	Exceeds Expectations ($\geq 90\%$)	Meets Expectations (70%-89%)	Developing (50%-69%)	Unsatisfactory ($< 50\%$)	Score
Performance (15)	Able to present full knowledge of both problem and solution.	Able to present adequate knowledge of both problem and solution	Able to present sufficient knowledge of both problem and solution	No or very less knowledge of both problem and solution	
Viva (5)	Able to answer the questions easily and correctly across the project.	Able to answer the questions related to the project	Able to answer the questions but with mistakes	Unable to answer the questions	
Obtained Score out of (20)					
Obtained Score out of (X)= Obtained Score out of (20) /20* X					

Final Lab Assessment

Criteria	Score
Laboratory Report(15)	
Semester Project (15)	
Laboratory Examination (20)	
Total Score out of (50)	

Examiner: _____
(Name and Signature)

Lab #1

Getting Started (Revision of C++)

Introduction to:

1. Recall all programming techniques
2. Use of Array, Structures, Functions & Pointers

Basic If-Else Statement

There are three types of control statements used in C language

- If else statements
- Switch statements
- Goto statements

```
if(<condition>){  
    //Execute the statements if<condition>is TRUE  
}  
Else if (<another condition> ){  
    //Execute the statements if<another condition>is TRUE and  
    //<condition>is FALSE  
}
```

Example:

```
include<iostream>    using  
namespace std;
```

```
int main()            //Most important part of the program!  
{ int age;  
  variable...
```

```
cout<<"Please input your age:";    //Asks for age  
cin>>age;  
// The input is put in age  
cin.ignore();  
// Throw away enter  
if ( age < 100 ) {  
    //If the age is less tha100  
    cout<<"You are pretty young!\n"; // Just to show you it  
    works...  
}  
else if ( age == 100 ) {  
    // Iuse else just to show an example  
    cout<<"You are old\n";  
    //Just to show you it works...
```

```
    } else  
{  
cout<<"You are really old\n";    //Executed if no other statement is  
}  
cin.get();  
  
}
```

Array

An array is a container object that holds a fixed number of values of a single type. The length of an array is established when the array is created. After creation, its length is fixed. You have seen an example of arrays already, in the main method of the "Hello World!" application. Example

```
#include <iostream>
```

```
using namespace std;
```

```
void printArray(int arg[], int length){  
    for (int n=0; n<length; ++n)  
        cout<<arg[n] <<" "; cout<<"\n";  
}
```

```
int main ()  
{  
    int firstArray[] = {5,10,15};  
    int secondArray[] = {2,4,6,8,10};  
    printArray(firstArray,3);  
    printArray(secondArray,5);  
}
```

2. Structures

A data structure is a group of data elements grouped together under one name. These data elements, known as members, can have different types and different lengths. Definition of Structure

```
struct<struct-type>{  
    <type><identifier_list>;  
    <type><identifier_list>;  
    ...  
};
```

Example

```
#include <iostream>  
using namespace std;
```

```
struct person {  
    char name[50];  
    int age; float  
    salary;  
};
```

```
int main() { person p1;
cout<<"Enter Full name:";
cin.get(p1.name, 50);
cout<<"Enter age:";
cin>> p1.age; cout<<"Enter
salary: ";
cin>>p1.salary;

cout<<"\n Displaying Information."<<endl;
cout<<"Name: "<< p1.name <<endl;
cout<<"Age: "<< p1.age <<endl;
cout<<"Salary: "<< p1.salary;

return 0;
}
```

OUTPUT

Enter Full name: Magdalena Dankova

Enter age: 27

Enter salary: 1024.4

Displaying Information.

Name: Magdalena Dankova

Age: 27

Salary:1024.4

```

feet: "; cin>>
(*ptr).feet;cout<<"Enter inch: "; cin>>
(*ptr).inch;

cout<<"Displaying information."<<endl;
cout<<"Distance="<< (*ptr).feet <<"feet"<< (*ptr).inch <<" inches";

return0;
}

```

OUTPUT

```

Enterfeet:4
Enter inch: 3.5
Displaying information.
Distance=4feet3.5inches

```

In this program, a pointer variable ptr and normal variable d of type structure Distance is defined. The address of variable d is stored to pointer variable, that is, ptr is pointing to variable d. Then the member function of variable d is accessed using pointer.

Note: Since pointer ptr is pointing to variable d in this program,(*ptr).inch and d.inch is exact same cell. Similarly, (*ptr).feet and d.feet is exact same cell.

Pointers: Implementation of Null pointer, Void Pointer, Invalid Pointer, Dangling Pointer Reference Variable

A **pointer** is a variable whose value is the address of another variable. Like any variable or constant, you must declare a pointer before you can work with it. The general form of a pointer variable declaration is –

```
type* var-name;
```

Here, **type** is the pointer's base type; it must be a valid C++ type and **var-name** is the name of the pointer variable. The asterisk you used to declare a pointer is the same asterisk that you use for multiplication. However, in this statement the asterisk is being used to designate a variable as a pointer. Following are the valid pointer declaration –

```

int *ip; // pointer to an integer
double *dp; //pointer to a double
float*fp; // pointer to a float
char*ch // pointer to character

```

The actual data type of the value of all pointers, whether integer, float, character, or otherwise, is the same, a long hexadecimal number that represents a memory address. The only difference between pointers of different data types is the data type of the variable or constant that the pointer points to.

Null Pointers

The NULL pointer is a constant with a value of zero defined in several standard libraries including iostream. Consider the following program

```
#include<iostream>

using namespace std;
int
main () {
    int*ptr=NULL;
    cout<<"The value of ptr is "<<ptr;

    return 0;
}
```

When the above code is compiled and executed, it produces the following result–

The value of ptr is 0

On most of the operating

Lab Tasks

1. Write a Program to enter three integers and output the smallest integers using IF.
2. Write a Program to enter 10 integers in a single-dimension array and then print out the array in ascending order.
3. Write a program to create structure named student. Take information of student from user as input (StdID, StdName, StdAge etc.) Display the output.
4. Perform above task using pointers to structure

Lab#2

Measurement of Time Complexity of an algorithm

Sometimes there are more than one way to solve a problem .We need to learn how to compare the performance different algorithms and choose the best one to solve a particular problem. While analyzing an algorithm, we mostly consider time complexity and space complexity. Time complexity of an algorithm quantifies the amount of time taken by an algorithm to run as a function of the length of the input. Similarly, Space complexity of an algorithm quantifies the amount of space or memory taken by an algorithm to run as a function of the length of the input.

Time and space complexity depends on lots of things like hardware, operating system, processors, etc. However, we don't consider any of these factors while analyzing the algorithm. We will only consider the execution time of an algorithm.

Lets start with a simple example .Suppose you are given an array A and an integer x and you have to find if x exists in array A.

Simple solution to this problem is traverse the whole array A and check if the any element is equal to x.

```
For i:1to length of A
if A[i] is equal to x
    return TRUE
return FALSE
```

Each of the operation in computer take approximately constant time .Let each operation Takes time. The number of lines of code executed is actually depends on the value of x. During analysis of algorithm, mostly we will consider worst case scenario, i.e., when x is not present in the array A. In the worst case, the **if** condition will run N times where N is the length of the array A. So in the worst case, total execution time will be $(N*c+c)$. $N*c$ for the **if** condition

And c for the **return** statement (ignoring some operations like assignment of i).

As we can see that the total time depends on the length of the array A. If the length of the array will increase the time of execution will also increase.

Order of growth is how the time of execution depends on the length of the input. In the above example, we can clearly see that the time of execution is linearly depends on the length of the array. Order of growth will help us to compute the running time with ease. We will ignore the lower order terms, since the lower order terms are relatively in significant for large input .We use different notation to describe limiting behavior of a function.

O-notation:

To denote asymptotic upper bound, we use O-notation. For a given function $g(n)$, we denote

By $O(g(n))$ (pronounced “big-oh “of g of n”) the set of functions:

$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq c * g(n) \text{ for all } n \geq n_0\}$

Ω -notation:

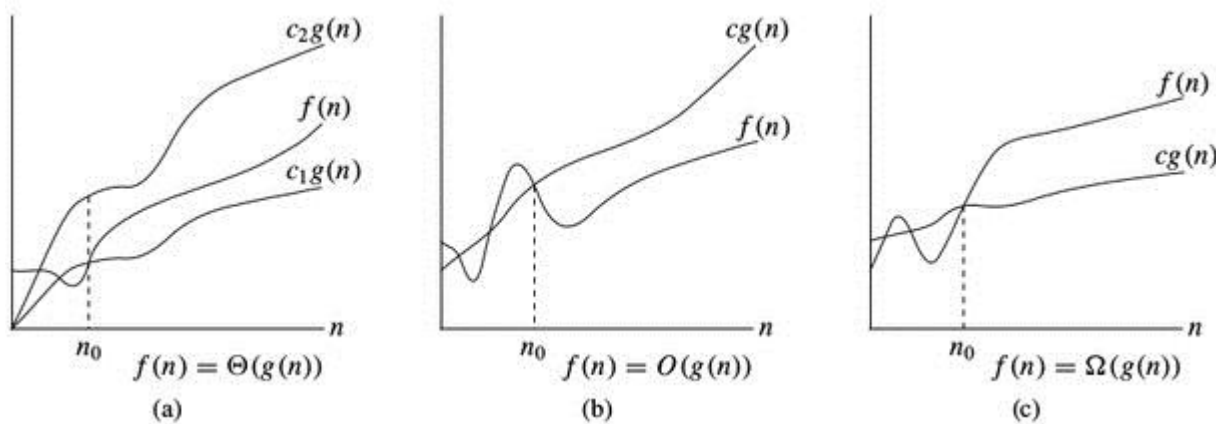
To denote asymptotic lower bound, we use Ω - notation. For a given function $g(n)$, we denote by $\Omega(g(n))$ (pronounced “big-omega of g of n”) the set of functions:

$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq c * g(n) \leq f(n) \text{ for all } n \geq n_0\}$

Θ -notation:

To denote asymptotic tight bound, we use Θ - notation. For a given function $g(n)$, we denote by $\Theta(g(n))$ (pronounced “big-theta of g of n”) the set of functions:

$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2 \text{ and } n_0 \text{ such that } 0 \leq c_1 * g(n) \leq f(n) \leq c_2 * g(n) \text{ for all } n \geq n_0\}$



Time complexity notations

While analyzing an algorithm, we mostly consider O -notation because it will give us an upper limit of the execution time i.e. the execution time in the worst case.

To compute O -notation we will ignore the lower order terms, since the lower order terms are relatively insignificant for large input.

Let $f(N) = 2 * N^2 + 3 * N + 5$

$O(f(N)) = O(2 * N^2 + 3 * N + 5) = O(N^2)$

Lets consider some example:

1.

```
Int count = 0;
for (int i = 0; i < N; i++)
    for(int j=0; j<i; j++)
        count++;
```

Lets see how many times **count++** will run.

When $i=0$, it will run 0 times.

When $i=1$, it will run 1 times.

When $i=2$, it will run 2 times and so on.

Total number of times **count++** will run is $0+1+2+\dots+(N-1) = N*(N-1)/2$. So the time complexity will be $O(N^2)$.

2.

```
Int count = 0;
for(int i=N; i>0; i/=2) for(int
    j =0; j <i; j++)
    count++;
```

This is a tricky case. In the first look, it seems like the complexity is $O(N*\log N)$. N for the j 's loop and $\log N$ for i 's loop. But it's wrong. Let's see why.

Think about how many times **count++** will run.

When $i=N$, it will run N times.

When $i=N/2$, it will run $N/2$ times.

When $i=N/4$, it will run $N/4$ times and so on.

Total number of times **count++** will run is $N+N/2+N/4+\dots+1=2*N$. So the time complexity will be $O(N)$.

Implementation for elementary sorting algorithms

Introduction to:

1. Sorting a static list of values using different Algorithms
2. Performance issue of sorting algorithms

Bubble Sort

Bubble sort is a simple and well-known sorting algorithm. It is used in practice once in a blue moon and its main application is to make an introduction to the sorting algorithms. Bubble sort belongs to $O(n^2)$ sorting algorithms, which makes it quite inefficient for sorting large data volumes. Bubble sort is stable and adaptive.

Algorithm

1. Compare each pair of adjacent elements from the beginning of an array and if they are in reversed order, swap them.
2. If at least one swap has been done, repeat step 1.

You can imagine that on every step big bubbles float to the surface and stay there. At the step, when no bubble moves, sorting stops. Let us see an example of sorting an array to make the idea of bubble sort clearer.

Example .Sort {5, 1, 12, -5, 16} using bubble sort.

Complexity analysis

Average and worst case complexity of bubble sort is $O(n^2)$. Also, it makes $O(n^2)$ swaps in the worst case. Bubble sort is adaptive. It means that for almost sorted array it gives $O(n)$ estimation. Avoid implementations, which don't check if the array is already sorted on every step (any swaps made). This check is necessary, in order to preserve adaptive property

Selection Sort

Selection sort is one of the $O(n^2)$ sorting algorithms which makes it quite inefficient for sorting large data volumes. Selection sort is notable for its programming simplicity and it can over perform other sorts in certain situations (see complexity analysis for more details).

Algorithm

The idea of algorithm is quite simple. Array is imaginary divided into two parts - sorted one and unsorted one. At the beginning, sorted part is empty while unsorted one contains whole array. At every step, algorithm finds minimal element in the unsorted part and adds it to the end of the sorted one. When unsorted part becomes empty, algorithm stops.

When algorithm sorts an array, it swaps first element of unsorted part with minimal element and then it is included to the sorted part. This implementation of selection sort is not stable. In case of linked list is sorted, and, instead of swaps, minimal element is linked to the unsorted part, selection sort is stable.

Let us see an example of sorting an array to make the idea of selection sort clearer.

Example: Sort {5, 1, 12, -5, 16, 2, 12, 14} using selection sort.

Complexity analysis

Selection sort stops, when unsorted part becomes empty. As we know, on every step number of unsorted elements decreased by one. Therefore, selection sort makes n steps (n is number of elements in array) of outer loop before stop. Every step of outer loop requires finding minimum in unsorted part. Summing up, $n + (n - 1) + (n - 2) + \dots + 1$, results in $O(n^2)$ number of comparisons. Number of swaps may vary from zero (in case of sorted array) to $n - 1$ (in case array was sorted in reversed order), which results in $O(n)$ number of swaps. Overall algorithm complexity is $O(n^2)$.

Fact that selection sort requires $n-1$ number of swaps at most makes it very efficient in situations, when write operation is significantly more expensive, than read operation.

Insertion Sort

Insertion sort belongs to the $O(n^2)$ sorting algorithms. Unlike many sorting algorithms with quadratic complexity, it is actually applied in practice for sorting small arrays of data. For instance it is used to improve quick sort routine. Some sources notice that people use same algorithm ordering items, for example, hand of cards.

Algorithm

Insertion sort algorithm somewhat resembles selection sort. Array is imaginary divided into two parts - sorted one and unsorted one. At the beginning, sorted part contains first element of the array and unsorted one contains the rest. At every step, algorithm takes first element in the unsorted part and inserts it to the right place of the sorted one. When unsorted part becomes empty, algorithm stops. Sketchy, insertion sort algorithm step looks like this:

Becomes

The idea of the sketch was originally posted here.

Let us see an example of insertion sort routine to make the idea of algorithm clearer.

Example. Sort {7, -5, 2, 16, 4} using insertion sort.

The ideas of insertion

The main operation of the algorithm is insertion. The task is to insert a value in to the sorted part of the array. Let us see the variants of how we can do it.

"Sifting down" using swaps

Lab Task

1 Create a program that take an array of 10 inputs from the user and generate the sorted out put using the following Algorithms;

- Bubble Sort
- Insertion Sort
- Selection Sort

Lab#3

Implementation of searching algorithms over an Array based List

Introduction to:

1. Different searching techniques using static Data Structure i.e Array
2. Efficiencies and deficiencies of both the searching techniques
3. Internals and Abstraction of matrices and their representation

Binary Search

Binary Search is applied on the sorted array or list. In binary search, we first compare the value with the elements in the middle position of the array. If the value is matched then we return the value. If the value is less than the middle element, then it must lie in the lower half of the array and if it's greater than the element then it must lie in the upper half of the array. We repeat this procedure on the lower (or upper) half of the array. Binary Search is useful when the rear large numbers of elements in an array.

Binary Search Algorithm

Step 1: Set the list to be the whole list

Step2: Find the middle value of the list

Step3: If the middle value is equal to the target then we declare victory and stop

Step4: If the middle item is less than the target then we set the new list to be the upper half of the old list and we repeat from Step 2 using the new list.

Step5: If the middle value is greater than the target, then we set the new list to be the bottom half of the list, and we repeat from Step 2 with the new list.

Linear Search

A linear search is the basic and simple search algorithm. A linear search searches an element or value from an array till the desired element or value is not found and it searches in a sequence order. It compares the element with all the other elements given in the list and if the element is

matched it returns the value index else it return -1. Linear Search is applied on the unsorted or unordered list when there are fewer elements in a list.

Linear Search Algorithm

Step1: Start with the first item in the list

Step2: Compare the current item to the target

Step3: If the current value matches the target then we declare victory and stop

Step4: If the current value is less than the target then set the current item to be the next to middle term and repeat from Step 2

Lab Tasks

1. Translate the linear search algorithm in to a program which either find the location LOC where ITEM appears in ARRAY or return LOC=0
2. Translate binary search and insertion algorithm in to a program which finds either the location LOC where ITEM appears in ARRAY or the location LOC where ITEM should be inserted into ARRAY. (For object 3, 4, and 5 take an array of 10 elements given below)
3. Write a program which uses Binary Search to search the elements 52, and 33 and print it.
4. Write a program which uses Binary Search to search the elements 45 and 78 and delete it.
NUMBERS=[11,22,33,36,45,52,57,60,64,78]

Lab#6

Implementation of linked list with the help of algorithms for Insertion, Deletion and Search an element

Introduction to:

- LINEDLIST
- Implementation of LINKEDLIST
- Operation of LINKEDLIST

Singly-linked list

Linked list is a very important dynamic data structure. Basically, there are two types of linked list, singly-linked list and doubly-linked list. In a singly-linked list every element contains some data and a link to the next element, which allows keeping the structure. On the other hand, every node in a doubly-linked list also contains a link to the previous node. Linked list can be an underlying data structure to implement stack, queue or sorted list

Example:

Sketchy, singly-linked list can be shown like this:

Each cell is called a node of a singly-linked list. First node is called head and it's a dedicated node. By knowing it, we can access every other node in the list. Sometimes, last node, called tail, is also stored in order to speed up add operation.

Operations on a singly-linked list

Concrete implementation of operations on the singly-linked list depends on the purpose, it is used for. Following the links below, you can find descriptions of the common concepts, proper for every implementation.

- Singly-linked list traversal
- Adding a node
- Removing a node

1. Singly-linked list. Traversal.

Assume that we have a list with some nodes. Traversal is the very basic operation, which presents as a part in almost every operation on a singly-linked list. For instance, an algorithm may traverse a singly-linked list to find a value, find a position for insertion, etc. For a singly-linked list, only forward direction traversal is possible.

Traversal algorithm

Beginning from the head,

1. check, if the end of a list has not been reached yet;
2. do some actions with the current node, which is specific for a particular algorithm;
3. Current node becomes previous and next node becomes current. Go to step 1.

Example

As for example, let us see an example of summing up values in a singly-linked list.

For some algorithms tracking the previous node is essential, but for some, like an example, it's unnecessary. We show a common case here and a concrete algorithm can be adjusted to meet its individual requirements.

2. Singly-linked list. Addition (insertion) operation.

Insertion into a singly-linked list has two special cases. It's insertion of a new node before the head (to the very beginning of the list) and after the tail (to the very end of the list). In any other case, a new node is inserted in the middle of the list and so, has a predecessor and successor in the list. There is a description of all these cases below.

Empty list case

When the list is empty, which is indicated by ($head == NULL$) condition, the insertion is quite simple. Algorithm sets both head and tail to point to the new node.

Add first

In this case, a new node is inserted right before the current head node. It can

be done in two steps:

1. Update the next link of a new node, to point to the current head node.
2. Update head link to point to the new node.

Add last

In this case, new node is inserted right after the current tail node.

It can be done in two steps:

1. Update the next link of the current tail node, to point to the new node.
2. Update tail link to point to the new node.

General case

In general case, new node is always inserted between two nodes, which are already in the list. Head and tail links are not updated in this case.

Such an insert can be done in two steps:

1. Update link of the "previous" node, to point to the new node.
2. Update link of the new node, to point to the "next" node.

Singly-linked list. Removal (deletion) operation.

There are four cases, which can occur while removing the node. These cases are similar to the cases in add operation. We have the same four situations, but the order of algorithm actions is opposite. Notice, that removal algorithm includes the disposal of the deleted node, which may be unnecessary in languages with automatic garbage collection (i.e., Java).

List has only one node

When list has only one node, which is indicated by the condition, that the head points to the same node as the tail, the removal is quite simple. Algorithm disposes the node, pointed by head (or tail) and sets both head and tail to NULL.

Remove first

In this case, first node (current head node) is removed from the list. It can be done in two steps:

1. Update head link to point to the node, next to the head.
2. Dispose removed node.

Remove last

In this case, last node (current tail node) is removed from the list. This operation is a bit more tricky than removing the first node because algorithm should find a node which is previous to the tail first.

It can be done in three steps:

1. Update tail link to point to the node before the tail. In order to find it list should be traversed first, beginning from the head.
2. Set next link of the new tail to NULL.
3. Dispose removed node.

General case

In general case node to be removed is always located between two list nodes. Head and tail links are not updated in this case.

Such a removal can be done in two steps:

1. Update next link of the previous node, to point to the next node, relative to the removed node.
2. Dispose removed node.

Singly –linked list. Internal representation.

Every node of a singly-linked list contains following information:

- A value(user's data);
- A link to the next element (auxiliary

data). Sketchy, it can be shown like this:

First node called head and no other node points to it. Link to the head is usually stored in the class which provides an interface to the resulting data structure. For empty list, head is set to NULL.

Also it makes sense to store a link to the last node called tail .Though no node in the list can be accessed from the tail (because we can move forward only), it can accelerate an add operation, when adding to the end of the list. When list is big, it reduces add operation complexity essentially, while memory overhead is insignificant. Below you can see another picture, which shows the whole singly-linked list internal representation:

Lab Task

1 Write a program as follows

-----Linked List implementation of LIST ADT-----

1. Insert an element at the end
2. Delete an element from end
3. Search an element
4. Display List
5. Is List Empty

Exit

